What is a DaemonSet in Kubernetes?

A **DaemonSet** ensures that **a copy of a Pod runs on *every* node (or selected nodes)** in a Kubernetes cluster.

One Pod per Node is the core guarantee.

If:

- a new node joins → **Pod is automatically created**
- a node is removed → **Pod is automatically deleted**

Why do we need DaemonSet?

Some workloads are **node-level services**, not application replicas.

Why Deployment/ReplicaSet is NOT suitable

Deployments:

- manage **replicas**, not **nodes**
- don't guarantee one pod per node
- cannot auto-scale with node count

DaemonSet solves this

Requirement	Deployment	DaemonSet
One pod per node	❌	✅
Auto-attach to new nodes	❌	✅
Node-level responsibility	❌	✅
Infra / platform workloads	❌	✅

Typical Use Cases

DaemonSets are used for **infrastructure-level components**.

1 **Logging agents**

- Fluentd
- Fluent Bit
- Filebeat

Need access to **node log files** (/var/log)

2 **Monitoring agents**

- Node Exporter
- Datadog agent
- New Relic agent

Need **node CPU, memory, disk metrics**

3 **Networking components**

- Calico node
- Cilium agent
- kube-proxy (historically)

Need to run **on every node to manage networking**

4 **Security & compliance**

- Falco
- CrowdStrike
- Sysdig

Need kernel / syscall visibility per node

5 Storage / CSI node plugins

- EBS CSI Node
- Ceph CSI Node

Attach volumes locally to nodes

How DaemonSet works internally

DaemonSet Controller (inside kube-controller-manager)

- Watches:
  - Nodes
  - DaemonSet objects
- For each node:
  - Evaluates **nodeSelector / affinity / taints**
  - If eligible → creates a Pod
  - If not eligible → deletes Pod

DaemonSet does NOT use ReplicaSet

Scheduling behavior

Unlike normal pods:

- **Scheduler is bypassed**
- Pod is **directly bound to a node**
- PodSpec contains:

spec:

nodeName: <node-name>

This guarantees **exactly one pod per node**

What happens when a new node joins?

1. Node registers with API server
2. DaemonSet controller detects new node
3. Controller creates a Pod for that node
4. Kubelet pulls image & runs Pod

Fully automatic

What happens when a node is cordoned or drained?

Cordon

- Node marked unschedulable
- ❗ **DaemonSet Pods STILL run**

Drain

- Normal Pods → evicted
- DaemonSet Pods → **NOT evicted by default**

To force:

kubectl drain node1 --ignore-daemonsets

DaemonSet Pod Scheduling Rules

DaemonSet respects:

1.nodeSelector

nodeSelector:

node-role.kubernetes.io/worker: ""

2. Node Affinity

requiredDuringSchedulingIgnoredDuringExecution:

3.Taints & Tolerations

DaemonSets **usually tolerate taints**:

tolerations:

- operator: Exists

This allows running on:

- master/control-plane nodes
- tainted infra nodes

DaemonSet Update Strategy (Rolling Updates)

DaemonSet supports **rolling updates**, but differently from Deployments.

Default strategy

```
updateStrategy:
  type: RollingUpdate
```

How rolling update works

- Updates **node by node**
- Deletes old pod
- Creates new pod on same node

Optional:

```
rollingUpdate:
  maxUnavailable: 1
```

There is **no maxSurge** (because 1 pod per node rule)

DaemonSet YAML Example (Fluent Bit)

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluent-bit
  namespace: kube-system
spec:
  selector:
    matchLabels:
      app: fluent-bit
  template:
    metadata:
      labels:
        app: fluent-bit
    spec:
      tolerations:
```

```
    - operator: Exists
    containers:
    - name: fluent-bit
      image: fluent/fluent-bit:latest
      volumeMounts:
      - name: varlog
        mountPath: /var/log
    volumes:

    - name: varlog
      hostPath:
        path: /var/log
```

Key points:

- hostPath → access node logs
- tolerations → runs on all nodes
- No replicas field

DaemonSet vs Deployment

Feature	DaemonSet	Deployment
Scaling	Node-based	Replica-based
Scheduler	Bypassed	Used
ReplicaSet	❌	✅
Use case	Infra agents	Applications
One pod per node	✅	❌

Advanced Scenario (Production)

Scenario: Logging in GKE / EKS

- Each node generates logs under /var/log/containers
- Fluent Bit DaemonSet:
    - Reads logs locally
    - Pushes to Elastic / Loki / GCS
- When cluster scales from 5 → 20 nodes:
    - 15 new Fluent Bit pods auto-created
    - No manual intervention

This is **why DaemonSet exists**

Common Mistakes

❌ Using Deployment for node agents
❌ Forgetting tolerations (agent won't run on master)
❌ Expecting HPA to work with DaemonSet
❌ Using DaemonSet for stateless apps

DaemonSet ensures exactly one pod per eligible node, mainly used for node-level infrastructure services like logging, monitoring, networking, and security agents. It bypasses the scheduler and is managed directly by the DaemonSet controller.